

P5: implementation of threads

In this laboratory assignment you will study a basic implementation of threads designed for a system with a single CPU. The implementation is carried out entirely in user space, without operating system support and without memory protection between threads. All threads share the same address space and may access any memory location, including their own control structures (Thread Control Block - TCB). As a consequence, an incorrect memory access caused by a programming error may corrupt the TCB and affect the full process.

Although this implementation does not rely on privileged instructions or special hardware registers (features required for process switching) the mechanism used to perform a context switch between threads is conceptually very similar to process context switching. In an operating system, additional steps would be required, such as switching page tables, enabling memory protection, or changing privilege levels. In this practice, since everything runs in user mode within a single memory space, the context switch is limited to saving and restoring the execution context (local variables and registers).

1. Previous knowledge

1.1. Cooperative Scheduling Model

The system follows a non-preemptive, or cooperative, scheduling model. A running thread keeps control of the CPU until it voluntarily yields it. This design has an important limitation: if a thread does not explicitly relinquish the processor, no other thread can execute, which may reduce responsiveness and fairness.

However, cooperative scheduling simplifies concurrency problems. Context switches can only occur at specific calls, such as to `thread_yield` or `cv_wait`. Unlike preemptive systems, where a switch may occur between any two instructions, here the programmer has precise control over when execution may change from one thread to another.

1.2. Memory Organization of a Process

To understand how threads are implemented, it is necessary to review how memory is organized during program execution. The memory of a process is divided into several logical segments.

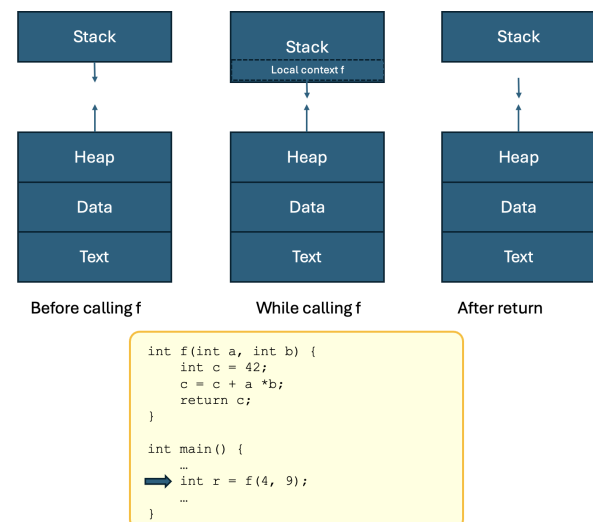
The text (or code) segment contains the program instructions and typically remains unchanged during execution, except when a new program is loaded using a function like `execvp`. The data segment stores global and static variables, whose size is fixed at program

startup. The heap is used for dynamic memory allocation through functions such as `malloc` and grows as memory is requested during execution. Finally, the stack stores the execution context of function calls (local variables, saved registers) and grows and shrinks dynamically as functions are invoked and return.

1.3 The Stack and Thread Execution Context

The stack is particularly important in this assignment because each thread has its own stack and register context. The stack contains local variables, function parameters, return addresses, and saved register values. Whenever a function is called, a new stack frame is created; when the function returns, that frame is removed. This dynamic behavior allows nested function calls and proper restoration of execution state. On the side you can find in the figure an schematic.

When switching from one thread to another, the current thread's stack pointer and register values must be saved, and the next thread's saved stack pointer and registers must be restored. Because each thread has its own stack, switching threads does not interfere with the local variables or call history of other threads.



2. Provided Code and Environment

You are provided with the complete code implementing user-level threads with condition variables, together with a producer–consumer example that demonstrates their use. The material originates from Egos2000, which is a toy operating system for a more advanced operating systems course based on a RISC-V environment executed in an emulator. In that course, students progressively build different parts of a complete operating system. For this introductory assignment, the implementation is already completed, and your task is to understand how it works. [Read their instructions](#), since it explains the process of context switching, which you will need to understand for this practice.

The project includes C source files as well as RISC-V assembly files (with extension `.s`) that implement the low-level context switching routines. Special attention should be given to the assembly implementation of the context switch, as it shows how thread registers are saved and restored. The code has to be run in an emulator, whose install instructions are below.

2.1 Thread Control Block (TCB)

A central component of the implementation is the Thread Control Block (TCB). The TCB is a data structure that stores all the information required to manage a thread, and restore it when it is halted. This typically includes the saved register values, the stack pointer, and the thread's state (whether it is ready to continue, or is waiting in a lock, or for an I/O operation..). It also contains the initial function that will be executed (the same one that is passed as a parameter in the `pthread_create`, and also its initial argument.

When a thread stops running, its execution context is saved into its TCB. When the scheduler selects that thread to run again, the saved context is restored so that execution continues from exactly the point where it previously stopped.

2.2 Thread Management Functions

The function `thread_init` initializes the threading system by preparing the TCB. The function `thread_create`, together with the assembly routine `ctx_start` and the helper function `ctx_entry`, is responsible for creating a new thread, allocating its stack, and initializing its execution context so that it begins running at the specified function.

The low-level context switch is implemented in the assembly function `ctx_switch`. This function saves the current thread's registers into its TCB and restores the registers of the next thread to run. The function `thread_yield` allows the running thread to voluntarily give up the CPU and triggers a context switch to another runnable thread.

2.3 Synchronization with Condition Variables

The implementation also provides a synchronization mechanism called a condition variable (`cv`), used to coordinate threads, particularly in the producer–consumer example. The available functions are `cv_init`, `cv_wait`, and `cv_signal`. A thread that calls `cv_wait` suspends its execution until another thread calls `cv_signal` on the same condition variable.

Installation

Step1: Setup the compiler and compile egos-2000

- 1) Setup your working directory and name it as **\$EGOS**.

```
export EGOS=/home/YOURHOME/egos
cd $EGOS
git clone -b thread https://github.com/yhzhang0128/egos-2000.git
```

- 2) Copy the files (queue.c, queue.h, thread.c, thread.h) to the egos-2000 folder.
- 3) Download the [pre-built binaries of the RISC-V GNU compiler toolchain](#) from xPack to **\$EGOS** and compile egos-2000.

```
cd $EGOS
tar -zxf xpack-riscv-none-elf-gcc-14.2.0-3-{linux,darwin}-{x64,arm64}.tar.gz
export PATH=$PATH:$EGOS/xpack-riscv-none-elf-gcc-14.2.0-3/bin
cd $EGOS/egos-2000
make
```

Step2: Run egos-2000 on the QEMU emulator

Download the [pre-built binaries of QEMU](#) from xPack.

```
cd $EGOS
tar -zxf xpack-qemu-riscv-8.2.2-1-xxxxxx.tar.gz
export PATH=$PATH:$EGOS/xpack-qemu-riscv-8.2.2-1-xxxxxx/bin
cd $EGOS/egos-2000
make qemu
```

The last line will activate the simulator, and make it run. You will see the output through the terminal. When you have finished executing, you will need to press *Ctrl +a*, followed by *x* to finish the execution.

Note that every time that you open a new terminal, you will need to rerun all the exports commands.